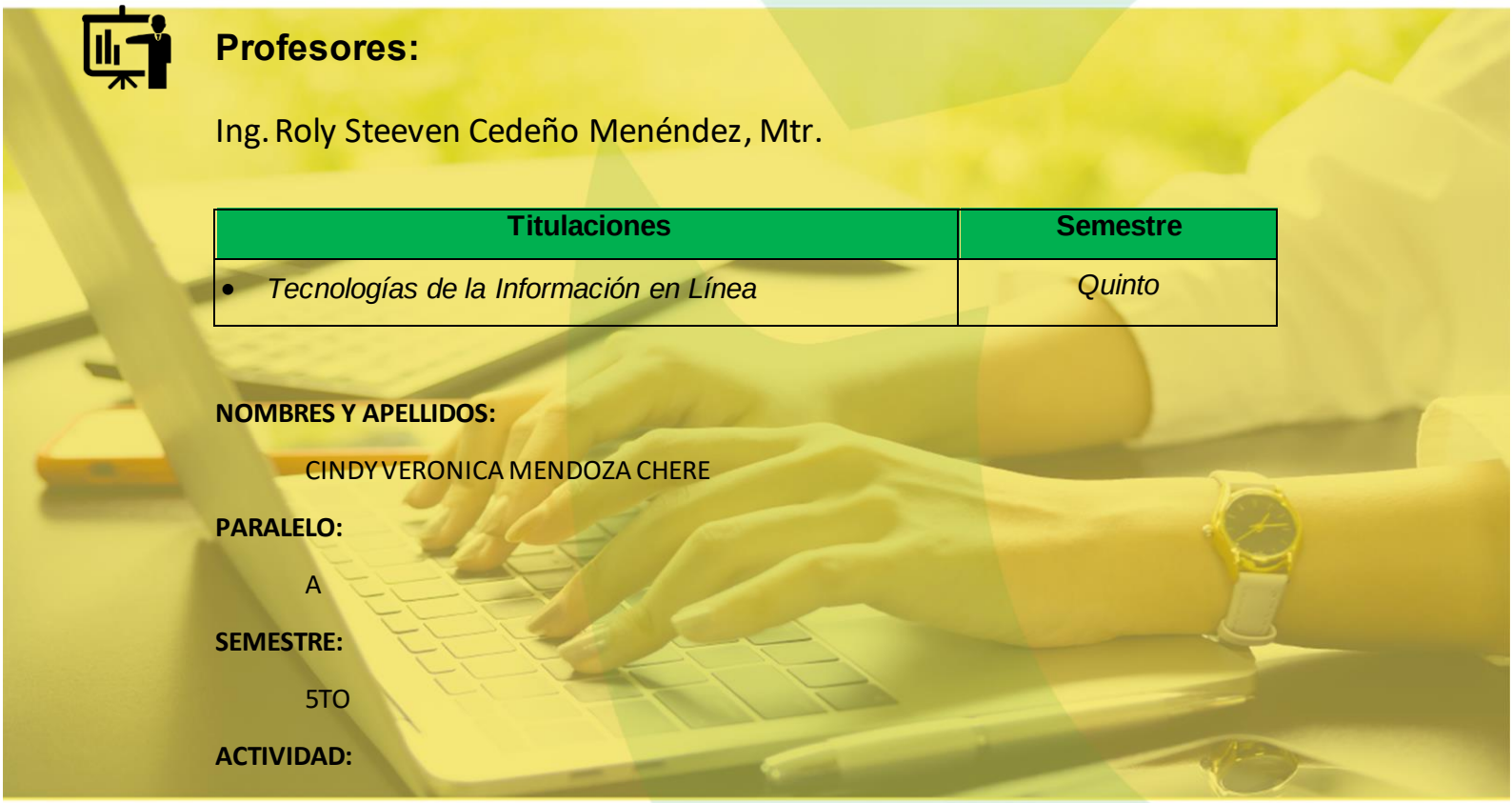
UTM online

Universidad Técnica de Manabí desde 1952

TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesores:

Ing. Roly Steeven Cedeño Menéndez, Mtr.

Titulaciones	Semestre
• *Tecnologías de la Información en Línea*	*Quinto*

NOMBRES Y APELLIDOS:

CINDY VERONICA MENDOZA CHERE

PARALELO:

A

SEMESTRE:

5TO

ACTIVIDAD:

1

PERIODO SEPTIEMBRE 2025 - ENERO 2026

Actividades a desarrollar en la Unidad 1

Actividad # 1

Sistema de Control de Rescate y Adopción Animales (SisCRA)

1. Análisis del problema

En la actualidad aun se puede evidenciar que muchos centros de refugio de animales, siguen usando un registro de datos manuales, mismos que no permiten llevar información netamente organizada y actualizada. Debido a este problema es necesario implementar un sistema de base de datos que permita gestionar de manera oportuna los casos de rescate, atención veterinaria y adopción, de esta manera evitar dificultades en el registro de los animales, asignación de responsables, seguimiento de tratamientos y la actualización de datos de los estados en cada caso.

Por esta razón se propone el diseño de un Sistema de Control de Rescate y Adopción de Animal, que permita llevar un registro automatizado de los animales rescatados, clasificación de especies, asignación de voluntarios o veterinarios y el seguimiento de expedientes dentro de las diferentes etapas del proceso.

Este sistema también permitirá organizar la información de manera estructurada, facilitar las notificaciones de los responsables y mantener un historial de mejoras que aporten a la trazabilidad, mantenimiento y evolución del software.

2. Definición de actores del sistema

Para el desarrollo del sistema los actores que interactuarán serán los siguientes:

Reportante

Persona que dará información del animal que necesita ayuda.

Acciones:

- Registra un reporte de rescate
- Da la ubicación del animal
- Detalla la condición
- Notifica al Veterinario

Veterinario

Responsable de atender el caso.

Acciones:

- Consultar cada caso asignado
- Verifica información del caso
- Registrar observaciones
- Actualizar el estado del animal
- Actualiza estado del caso
- Notifica cambios de estado
- Registrar información de la atención.
- Registrar historial médico.

Administrador

Persona encargada de coordinar y supervisar la gestión de los casos.

Acciones:

- Revisar los reportes recibidos
- Clasificar los animales
- Asignar veterinarios
- Supervisar expedientes
- Gestionar el proceso de adopción

3. Requerimientos del sistema

De conformidad con lo que el sistema va a realizar se han determinado los siguientes requerimientos, divididos en funcionales y no funcionales.

3.1. Funcionales

Requerimiento	Descripción
Registrar rescate	Permite crear un caso con los datos del animal, fecha, ubicación y descripción.
Clasificar la especie	Determinar si es perro, gato, o fauna silvestre.
Elaborar expediente	Crea un expediente asociado al caso de rescate según la especie.

Asignar responsables	El administrador asigna un veterinario.
Actualización de datos	Registra el proceso del cada caso: rescatado, tratamiento, disponible para adopción y adoptado.
Notificación de responsable	Emite una notificación cuando ocurre un nuevo rescate.
Consulta de expediente	Verificación de datos e historial de cada caso.

3.2. No Funcionales

Requerimiento	Descripción
Escalabilidad	Permite ingresar mas usuarios, animales y casos sin tener que rediseñar todo el sistema.
Mantenibilidad	Permite modificaciones y correcciones al software
Trazabilidad	Mantiene el historial de cambios mediante Git y GitHub.

4. Diseño del sistema

En este apartado se mostrará la creación de un repositorio en GitHub, un diagrama de casos de uso, diagrama de clases UML y la arquitectura general del sistema integrando el flujo de versiones.

4.1. Diagrama de casos de uso

El diagrama contestara la pregunta: *¿Quién puede hacer y que debe hacer dentro del sistema?*. Figura 1

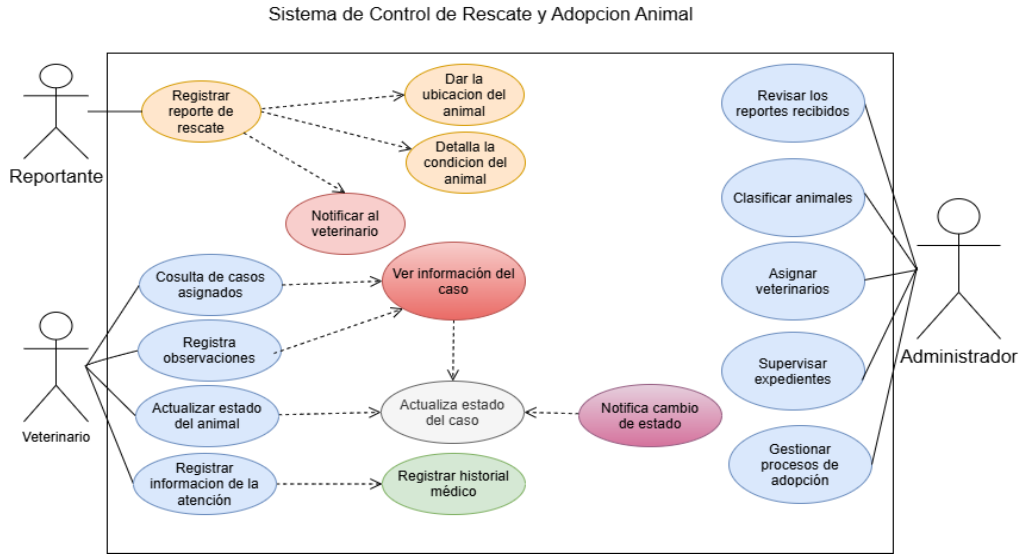


Figura 1: Casos de Uso

4.2. Diagrama de clases UML

En este apartado se muestra como estará organizado el sistema por dentro: clases, atributos, métodos y relaciones. Figura 2

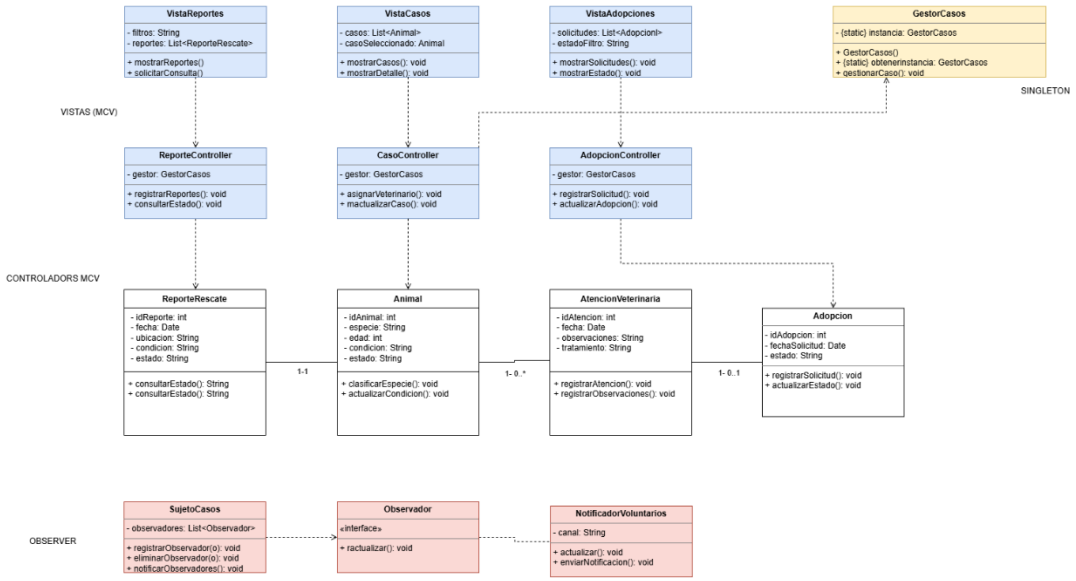


Figura 2: Diagrama de clases UML

4.3. Justificación

En la realización del diagrama UML, se hizo uso de tres patrones de diseño:

- **MVC (Modelo-Vista-Controlador)**

Siendo un patrón que se usa para organizar el sistema en tres componentes principales como son: Modelo, Vista y Controlador, representadas en el diagrama por las clases VistaReportes, VistaCasos y VistaAdopciones, las que permiten mostrar la información relacionada con los reportes de rescate, los casos de animales y las solicitudes de adopción. Los controladores ReporteController, CasoController y AdopcionController gestionan las acciones del usuario y coordinan las operaciones del sistema. Por su parte, las clases del modelo, como ReporteRescate, Animal, AtencionVeterinaria y Adopcion, representan las entidades principales del negocio.

Por esta razón se ha seleccionado el patron MVC, porque facilita el mantenimiento, organización y ampliación del sistema, al separa las responsabilidades, es posible modificar la interfaz de usuario sin alterar las entidades del modelo, favoreciendo la reutilización del código y el cumplimiento del principio de responsabilidad única de SOLID.

- **Singleton**

Garantiza que una clase tenga una instancia compartida y proporciona un punto de acceso global a ella. En el diagrama representando a las clases, GestorCasos, que contiene el atributo estático instancia: GestorCasos, un constructor privado GestorCasos() y el método estático obtenerInstancia(): GestorCasos. Su función es centralizar la gestión de los casos de rescate y facilitar el acceso a las operaciones comunes del sistema.

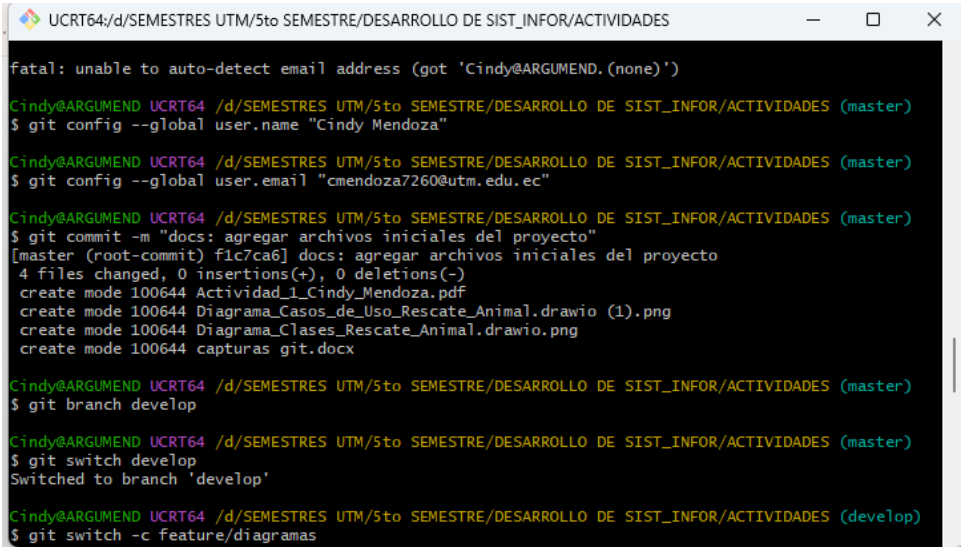
Se considero su aplicación para disponer de una instancia centralizada de GestorCasos, evitando la creación de múltiples gestores independientes que puedan ocasionar inconsistencias en la administración de los casos, permitiendo a los controladores el acceso al mismo gestor para coordinar operaciones relacionadas con los reportes, los animales y las adopciones.

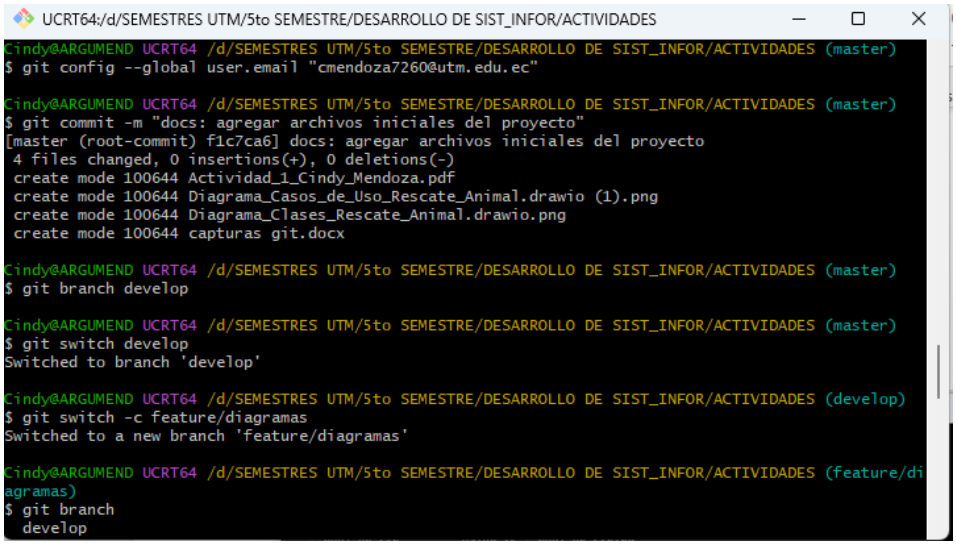
- **Observer**

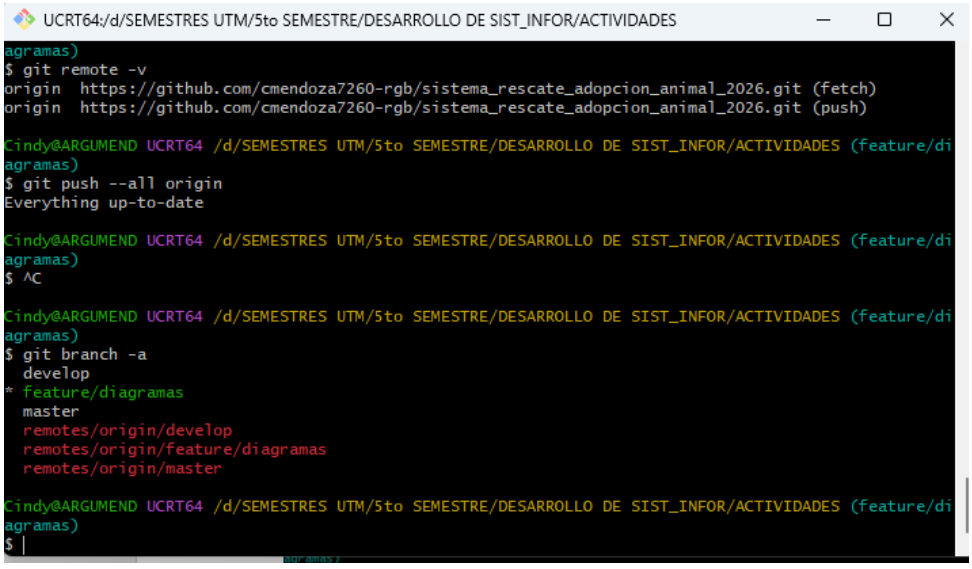
Al ser un patrón de diseño que permite informar a los voluntarios cuando se producen cambios importantes en cada caso de rescate, como la actualización, el estado animal o la disponibilidad de una nueva intervención. Facilitando así la comunicación entre los componentes del sistema.

En el diagrama UML, está representado por las clases SujetoCasos, Observador y NotificadorVoluntarios. La clase SujetoCasos mantiene una lista de observadores y dispone de los métodos registrarObservador(), eliminarObservador() y notificarObservadores(). La interfaz Observador define el método actualizar(), mientras que NotificadorVoluntarios implementa el comportamiento necesario para recibir actualizaciones y enviar notificaciones a los voluntarios.

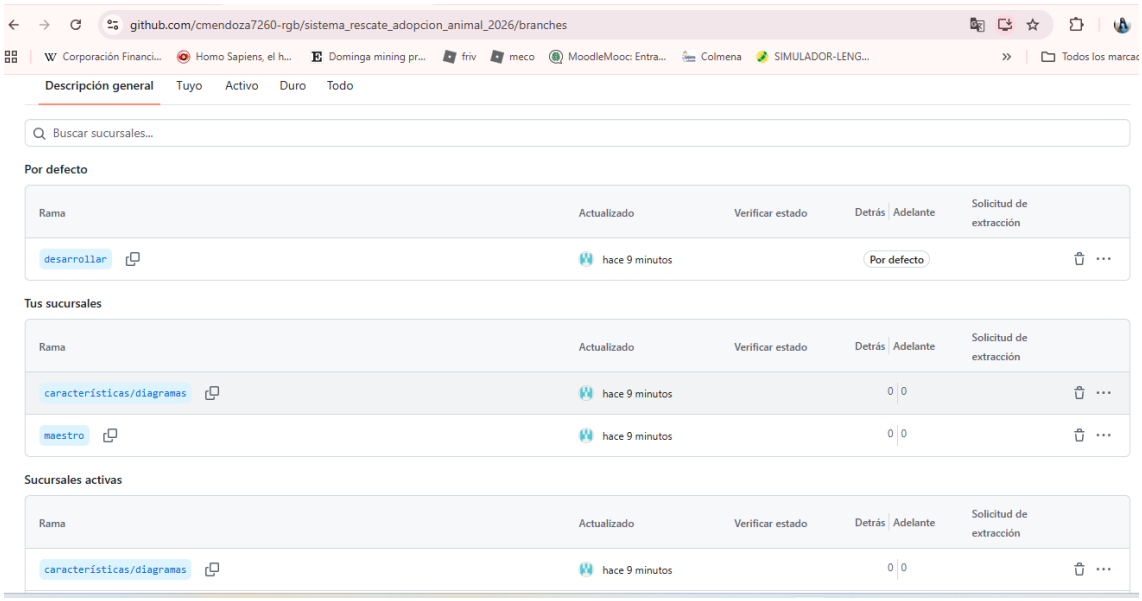
4.4. Capturas del historial de GitHub







4.5. Ramas en GitHub.



4.6. Los archivos publicados en el repositorio.

